SLO-04: SLO Alerting

> **Series:** SLO — Service Level Objectives | **Notebook:** 4 of 6 | **Created:** June 2026 | **Last Updated:** 10/12/2026

Overview

The highest-value alert a platform can send is "you are about to breach an SLO." This notebook covers **burn-rate alerting** — why it beats thresholding the raw SLI, how fast-burn and slow-burn multiwindow alerts work, how Dynatrace surfaces SLO alerts, and how to route them through Workflows without drowning the team in noise.

Table of Contents

1. Why Burn-Rate, Not Threshold-on-SLI
2. Fast-Burn and Slow-Burn
3. Configuring SLO Alerts in Dynatrace
4. Routing SLO Breaches
5. Avoiding Alert Fatigue

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with the SLO app and AutomationEngine (Workflows)
Permissions	`settings:objects:read/write`, workflow authoring permissions

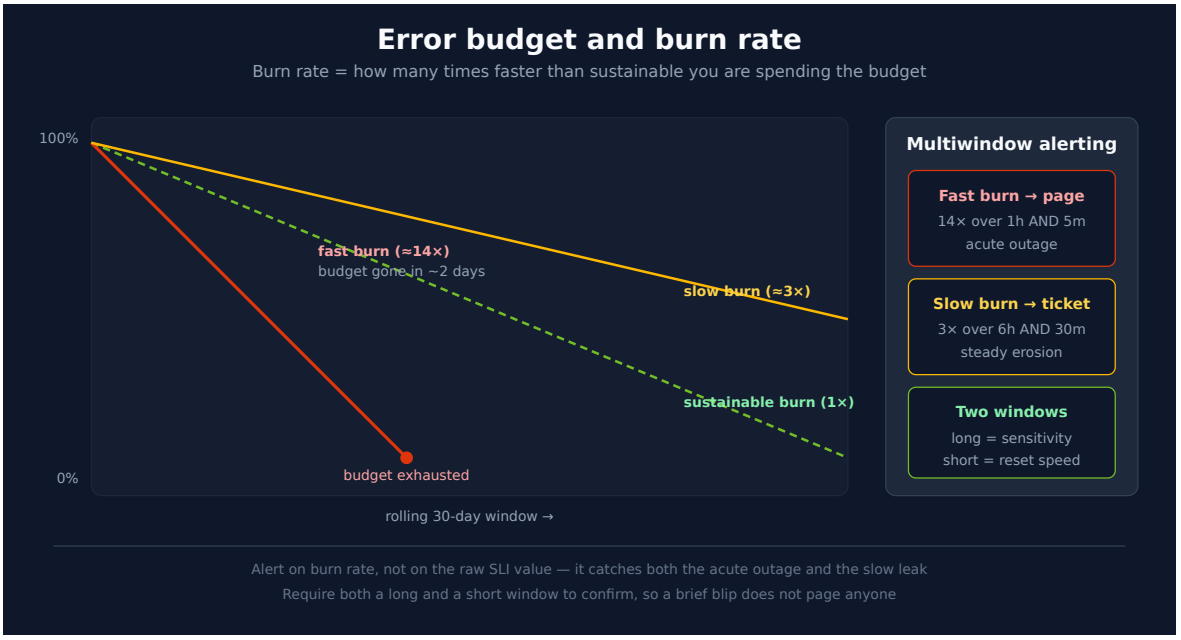
Requirement	Details
Prior reading	SLO-03 (error budgets and burn rate)

1. Why Burn-Rate, Not Threshold-on-SLI

The naive approach — alert when the SLI drops below the target — is a bad signal:

- **It fires late on slow leaks.** A rolling-30-day SLI barely moves when a service degrades for an hour, so the alert comes long after the damage.
- **It flaps on momentary dips.** A single bad interval can cross the line and recover, paging someone for nothing.

Burn rate fixes both. Because it measures the *rate* of budget consumption over a short window, a sharp degradation produces a high burn rate immediately, while a brief blip does not sustain. You alert on "the budget is draining too fast," which is what you actually care about.



2. Fast-Burn and Slow-Burn

The Google SRE Workbook's multiwindow, multi-burn-rate pattern requires **two windows** to agree before an alert fires. Its recommended parameters for a 99.9% SLO, reduced to the two tiers this notebook routes (page and ticket):

Tier	Burn rate	Windows (long AND short)	Budget consumed in the long window	Response
Fast burn	14.4×	1 hour AND 5 minutes	2%	Page — acute, budget gone in ~2 days
Slow burn	1×	3 days AND 6 hours	10%	Ticket — steady erosion

The Workbook's table also carries a middle **page** tier — 6× over 6 hours AND 30 minutes (5% consumed) — for incidents too slow for the 1-hour window but too costly to leave to a ticket. Add it if your fast-burn tier alone misses those.

The **long window** gives sensitivity (it confirms the burn is real); the **short window** gives fast reset (it stops alerting quickly once the burn stops). Requiring both prevents a momentary spike from paging anyone. The query below evaluates a fast-burn condition; wire it into a detector (Section 3):

```
// Fast-burn check: is the 1h burn rate &gt;= 14x against a 99.5% target?
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, from:-1h, interval:5m
| fieldsAdd bad_ratio = failures[] / total[]
| fieldsAdd observed_bad = arrayAvg(bad_ratio)
| fieldsAdd burn_rate = observed_bad / (1.0 - 0.995)
| fieldsAdd fast_burn = if(burn_rate &gt;= 14, 1, else: 0)
| fields burn_rate, fast_burn
```

3. Configuring SLO Alerts in Dynatrace

> **Announced, then withdrawn (SaaS 1.347).** An early version of the SaaS 1.347 release notes announced native multi-window burn-rate alerts for SLOs — the AND-across-two-windows condition §2 teaches you to build. Dynatrace took the item out of the notes on 09/08/2026, the day the rollout began: the changelog embedded in the release-notes page lists "Burn rate alerting for SLOs" among its removals, and as of 09/18/2026 the notes no longer mention burn-rate alerting at all.
>
> **Build the manual path.** The SLO documentation still describes only the manual route — a `burnRate` field added to the SLI plus Anomaly Detection with a -1h look-

> back — which is what §2 and the rest of §3 build. If native multi-window alerting ships in a later release, it will appear in that release's notes, and this section will change then. Until it does, do not plan around it.

Dynatrace surfaces SLO health as events you can alert on. The SLO documentation (re-read 09/28/2026) describes one route:

- **Anomaly Detection on the burn-rate query** — when you want the full multiwindow fast/slow-burn logic, evaluate a burn-rate query in the Anomaly Detection app and let it raise a Davis event. See AIOPS-02 §4 for the detector build flow and event template.

It does not describe an alert raised by the SLO object itself, so build on the Anomaly Detection route. The resulting event carries the SLO context you set in its event properties — which is what a workflow routes on.

> **Where the burn-rate custom alert is created (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and the Anomaly Detection route above is built there.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

The documented recipe for the burn-rate query is to append two lines to the SLI definition, then feed that into Anomaly Detection:

```
| fieldsAdd sli = "YOUR SLI"
| fieldsAdd target = "YOUR SLO-target" // in percentage
| fieldsAdd burnRate = ((100 - sli[]) / (100 - target))
| fieldsRemove sli
```

If you want one aggregated burn-rate value across every contributing entity rather than a per-entity series (at the cost of losing which entity is driving it), add a `summarize sloBurnRate = avg(burnRate[]), timeframe = takeFirst(timeframe), interval = takeFirst(interval)` step. Segments are **not currently supported** inside Anomaly Detection, so a segment-scoped SLO's burn-rate alert needs the segment expressed as a query filter instead.

Fast-burn tuning, per Dynatrace's own recommendation: Anomaly Detection's **-1h look-back window** is called out as well-suited for fast-burn alerting, with **10–14** as "a good starting point" for the static burn-rate threshold at that window — adjust up or down based on the SLO's criticality and evaluation period. Dynatrace also recommends four event properties on the custom alert so the resulting event carries enough context to route and correlate automatically: `dt.source` **entity** (attaches the affected service entities), `event.type` (`ERROR_EVENT` / `AVAILABILITY_EVENT` / `PERFORMANCE_EVENT` — matches the event into Dynatrace Intelligence RCA), `slo.name` (ties the event back to the SLO, which is unique per environment), and `dt.owner` (drives automatic routing to the right team).

> The **slow-burn tier** in §2 (longer look-back, lower threshold, ticket-not-page routing) comes from the Google SRE Workbook, not from Dynatrace. As of 09/28/2026 the Dynatrace SLO documentation — the Service-Level Objectives page and its subpages — gives a numeric recommendation only for fast burn (the -1h look-back, 10–14); no slow-burn window or threshold appears there. Treat the Workbook values as a starting point to tune against your own traffic and SLO criticality rather than a vendor-specified default, and confirm your detector can evaluate the longer slow-burn windows before relying on them.
>
> **Sources:** What's new in Dynatrace SaaS 1.347 (DT docs) — checked 09/18/2026: no burn-rate alerting item remains; see the page's changelog entry for 09/08/2026. Service-Level Objectives (DT docs) — read 09/18/2026, still documenting only the manual `burnRate` + Anomaly Detection route.

4. Routing SLO Breaches

A burn-rate custom alert raises a Davis event, which Dynatrace Intelligence considers when it detects problems. Route it with a workflow on that event, or on the problem it joins. Routing is the WFLOW series' job, not a separate mechanism:

1. **Workflow trigger** filters for the burn-rate event, or the problem it joins, on the event properties you set on the custom alert: `slo.name` and `dt.owner`.
2. **Route to the owning team's channel** — Slack, Teams, PagerDuty, or an ITSM incident.
3. **Differentiate by tier** — fast-burn → page (PagerDuty / on-call); slow-burn → ticket (Jira / ServiceNow).

This is exactly the routing covered in WFLOW-04. The key dependency is upstream: the burn-rate detector's **event template must carry the team/service metadata** (AIOPS-02 §4) or the workflow has nothing to filter on.

> **Sources:** Service-Level Objectives (DT docs) — *"an event with the set event properties is raised"*; `dt.owner` is *"a team identifier that allows automatic routing and ticketing"*. Read 10/12/2026.

5. Avoiding Alert Fatigue

- **Only page on fast burn.** Slow burn is a ticket, not a 3 a.m. call.
- **Require both windows.** The short window is what stops the alert from re-firing as conditions recover.
- **Set recovery/dealerting deliberately.** Use the detector's `dealertingSamples` (AIOPS-02 §4) so an alert closes cleanly rather than flapping.
- **One SLO breach, one notification.** Let the problem group related signals; do not also alert on the underlying raw metrics or you double-notify.
- **Review what fired.** If an SLO pages and no one acts, the target or the burn thresholds are wrong — tune them.

> **Sources:** Service-Level Objectives (DT docs) — burn-rate DQL append pattern, -1h/10–14 fast-burn recommendation, and the four recommended event properties are cited verbatim from the "Monitor error-budget burn rates" section (fetched 07/01/2026, re-read 09/28/2026); Site Reliability Engineering — Alerting on SLOs (Google SRE Workbook) — the §2 window pairs are from its *"Recommended parameters for a 99.9% SLO alerting configuration"* table. Fast-burn query re-executed against a live tenant 08/28/2026 (burn_rate 9.05, fast_burn 0 — below the 14x trigger, which is the expected shape for a healthy window). The fast-burn-only scope of the Dynatrace recommendation is a dated reading (09/28/2026) of the Service-Level Objectives page and its seven subpages, none of which gives a slow-burn window or threshold.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace

documentation.